In [208]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
```

Problem 2 - Part 1 - Visualization

Load & Process Data

In [2]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_unsupervised\\'
X = np.load(path + 'X.npy')
```

In [3]:
```
X_log = np.log2(X+1)

print('Dataset : Gene Expression Matrix')
print('------------------------------------------------------------')
print('(Nb_Cells, Nb_Genes) = ', X.shape, '      (rows x columns)')
print('Min_Value , Max_Value = ', round(np.min(X),0), ',' , round(np.max(X),0))
print('------------------------------------------------------------')
print('Apply log-transform to reduce this extremely stretched range:')
print('X_log <- np.log2(X+1)')
print('Min_Value , Max_Value = ', round(np.min(X_log),0), ',' , round(np.max(X_log),0))
```

```
Dataset : Gene Expression Matrix
------------------------------------------------------------
(Nb_Cells, Nb_Genes) =  (2169, 45768)       (rows x columns)
Min_Value , Max_Value =  0.0 , 341222.0
------------------------------------------------------------
Apply log-transform to reduce this extremely stretched range:
X_log <- np.log2(X+1)
Min_Value , Max_Value =  0.0 , 18.0
```

1. (3 points) Visualization of the three brain cell types

- At first, let's reduce the dimensionality of the features from 45,768 genes down to a number of meaningful dimensions.
- We use PCA to keep only a tractable number of dimensions (in simple terms we keep the directions with maximum variance out of a subset of orthogonal axes).

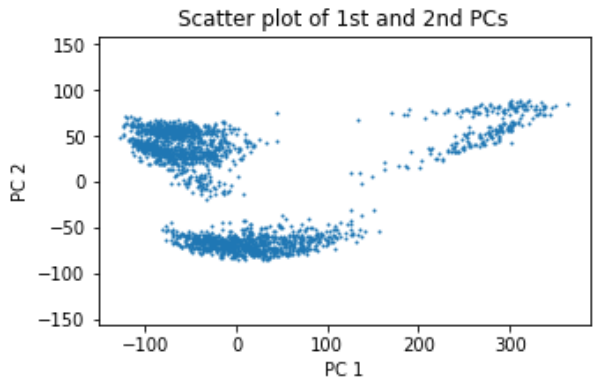
PCA

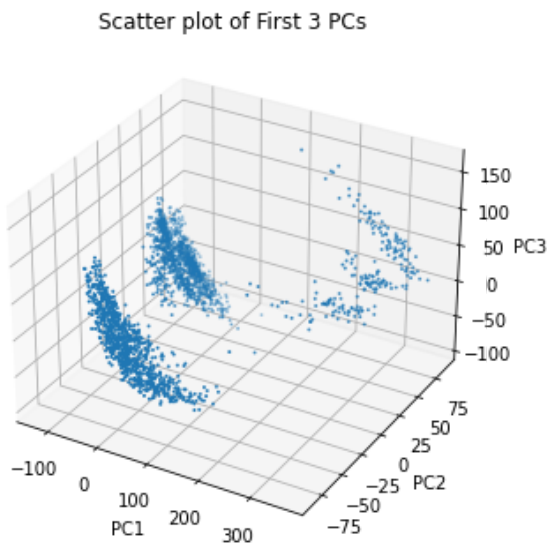
In [217]:
```
#######
# PCA #
#######

pca_log = PCA(random_state = 0)
Z_log = pca_log.fit_transform(X_log)
# A note on chosen parameters for sklearn.decomposition.PCA:
# Specified:
# - Random_state for reproducible results
# Unspecified (default values):
# - svd_solver = auto (randomized method is used given the dimension > 500x500)
# - n_components = min(n_samples, n_features) = n_samples = 2,169
```

In [131]:
```
print('Principal Components dimension: ', Z_log.shape)
print('Variance explained by first 50 PCs: ', round(np.sum(pca_log.explained_variance_ratio_[0:50
]),3))
print('-------------------------------------------------------------')
print('Variance explained by each of the top 10 PCs:')
print(np.round(pca_log.explained_variance_ratio_[0:10]*100,2))
```

```
Principal Components dimension:  (2169, 2169)
Variance explained by first 50 PCs:  0.338
-------------------------------------------------------------
Variance explained by each of the top 10 PCs:
[15.49  4.43  1.76  1.14  0.9   0.88  0.71  0.64  0.6   0.58]
```

In [184]:
```
#####################
# PCA Visualization #
#####################

plt.figure(figsize=(5,3))
plt.scatter(Z_log[:,0],Z_log[:,1], s=1)
plt.title("Scatter plot of 1st and 2nd PCs")
plt.xlabel("PC 1"); plt.ylabel("PC 2"); plt.axis("equal")

fig = plt.figure()
ax = Axes3D(fig)
ax.scatter(Z_log[:,0], Z_log[:,1], Z_log[:,2], s=1)
ax.set_xlabel('PC1'); ax.set_ylabel('PC2'); ax.set_zlabel('PC3')
ax.set_title("Scatter plot of First 3 PCs")
plt.show()
```



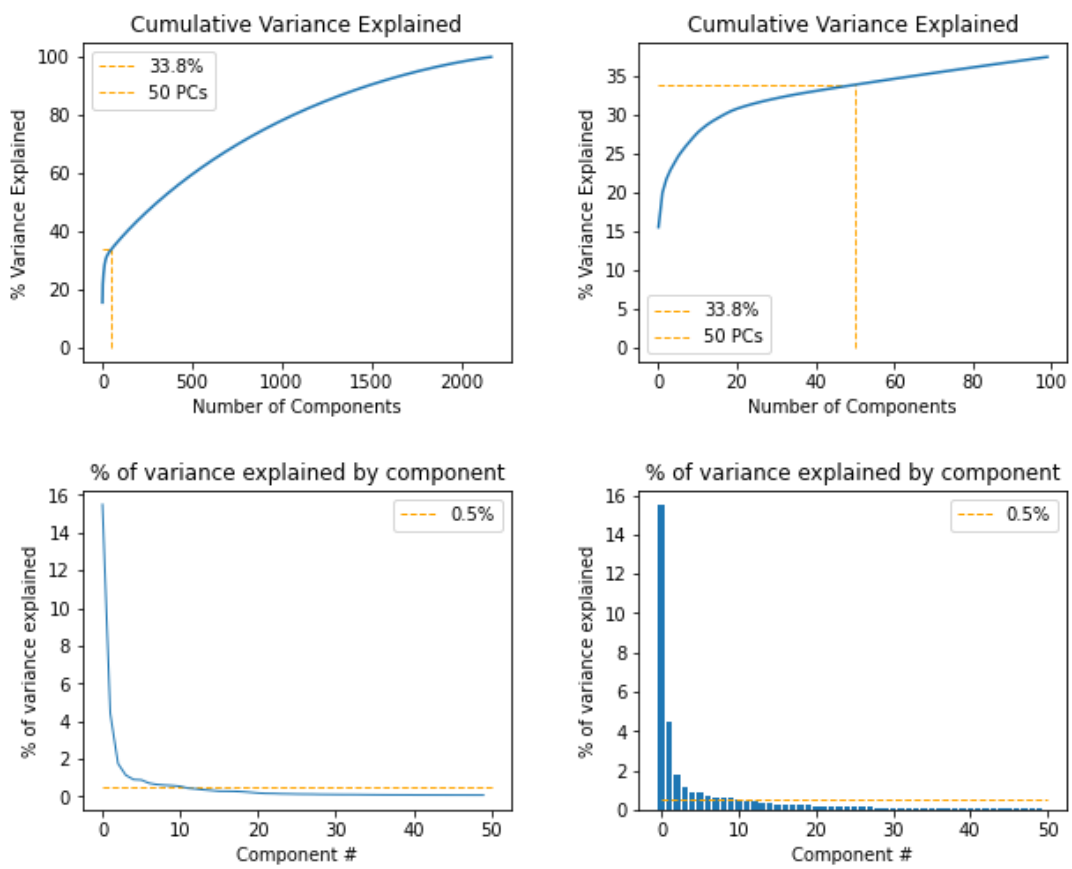


- The projection of the data onto the first two principal components (first graph above) shows <u>three distinct groups of cells that differ greatly</u>. The same interpretation can be drawn from the 3D projection onto the first three PCs (second graph).
- Though not as clear visually, we can guess that a subgroup classification exists.

In the following we'll continue the exploration of groups based only on a selection of principal components (for CPU usage).

In [113]:

```
##################
# Nb of PCs kept #
##################

plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.plot(np.arange(0,2169),np.cumsum(pca_log.explained_variance_ratio_*100))
plt.hlines(y=33.8, xmin=0, xmax=50, color='orange', linestyles='dashed', linewidth=1, label='33.8%'
)
plt.vlines(x=50, ymin=0, ymax=33.8, color='orange', linestyles='dashed', linewidth=1, label='50 PC
s')
plt.title("Cumulative Variance Explained"); plt.xlabel("Number of Components"); plt.ylabel("% Varia
nce Explained"); plt.legend()

plt.subplot(2,2,2)
plt.plot(np.arange(0,100),np.cumsum(pca_log.explained_variance_ratio_[0:100]*100))
plt.hlines(y=33.8, xmin=0, xmax=50, color='orange', linestyles='dashed', linewidth=1, label='33.8%'
)
plt.vlines(x=50, ymin=0, ymax=33.8, color='orange', linestyles='dashed', linewidth=1, label='50 PC
s')
plt.title("Cumulative Variance Explained"); plt.xlabel("Number of Components"); plt.ylabel("% Varia
nce Explained"); plt.legend()

plt.subplot(2,2,3)
plt.plot(np.arange(0,50),pca_log.explained_variance_ratio_[0:50]*100, linewidth=1)
plt.hlines(y=0.5, xmin=0, xmax=50, color='orange', linestyles='dashed', linewidth=1, label='0.5%')
plt.title("% of variance explained by component"); plt.xlabel("Component #"); plt.ylabel("% of vari
ance explained"); plt.legend()

plt.subplot(2,2,4)
plt.bar(np.arange(0,50), pca_log.explained_variance_ratio_[0:50]*100)
plt.hlines(y=0.5, xmin=0, xmax=50, color='orange', linestyles='dashed', linewidth=1, label='0.5%')
plt.title("% of variance explained by component"); plt.xlabel("Component #"); plt.ylabel("% of vari
ance explained"); plt.legend()

plt.subplots_adjust(left=None, bottom=None, right=None, top=None, wspace=0.3, hspace=0.4)
```

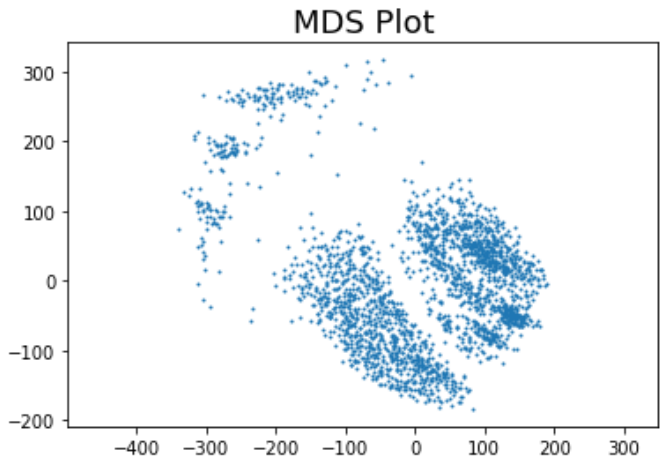


- The first 7 principal components explain 25% of variance. Though, in order not to reduced dimensions too much at this stage, we will use the 50 first PCs which explain nearly 34% of variance (and keep PCU usage under control).

In [189]:
```
# We keep only 50 PCs
Z_log_50 = Z_log[:,:50]
```

MDS

In [216]:
```
#######
# MDS #
#######

mds = MDS(n_components=2,verbose=1,eps=1e-5)
mds.fit(Z_log_50)
plt.scatter(mds.embedding_[:,0],mds.embedding_[:,1], s=1)
plt.title("MDS Plot",size=18)
plt.axis("equal")
plt.show()
```



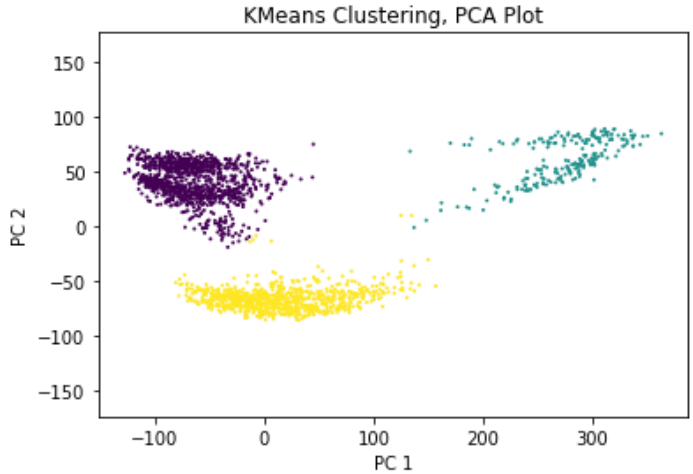
- MDS, which preserves distances, confirms what we've observed with PCA: at least 3 distinct and distant clusters can be formed from the data.

2. (4 points) Visualization of cells sub-types

- Let's firstly assign the cells to 3 groups using K-Means algorithm.
- Then we'll visualize the cells projected onto the top 50 principal components using T-SNE.
- T-SNE clearly shows subtypes of cells within each group.

In [212]:
```
##########
# KMeans #
##########
kmeans = KMeans(n_clusters=3,n_init=100, random_state = 0)
y = kmeans.fit_predict(Z_log_50)
```

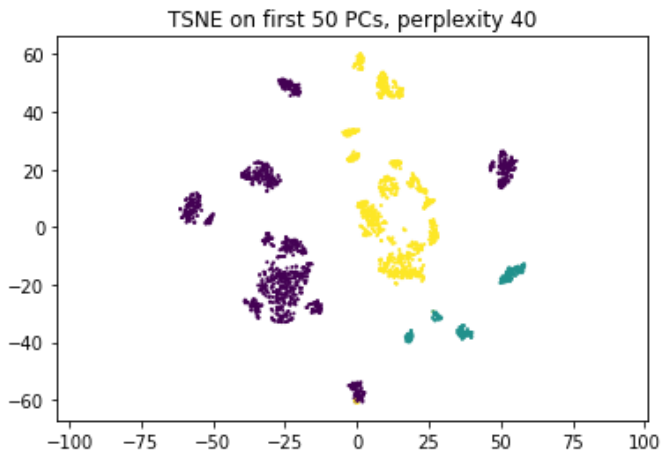
In [213]:
```
plt.scatter(Z_log_50[:,0],Z_log_50[:,1],c=y, s=1)
plt.title("KMeans Clustering, PCA Plot"); plt.xlabel("PC 1"); plt.ylabel("PC 2")
plt.axis("equal")
plt.show()
```



T-SNE

In [215]:
```
tsne = TSNE(n_components=2,verbose=1,perplexity=40, random_state=0)
Z_tsne = tsne.fit_transform(Z_log_50)
plt.scatter(Z_tsne[:,0],Z_tsne[:,1], c=y, s=1)
plt.title("TSNE on first 50 PCs, perplexity 40")
plt.axis("equal")
plt.show()
```

```
[t-SNE] Computing 121 nearest neighbors...
[t-SNE] Indexed 2169 samples in 0.026s...
[t-SNE] Computed neighbors for 2169 samples in 0.373s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2169
[t-SNE] Computed conditional probabilities for sample 2000 / 2169
[t-SNE] Computed conditional probabilities for sample 2169 / 2169
[t-SNE] Mean sigma: 20.955963
[t-SNE] KL divergence after 250 iterations with early exaggeration: 55.334656
[t-SNE] KL divergence after 1000 iterations: 0.490110
```



- Subtypes of cells (different sub-groups belonging to the three groups of cells) can be clearly distinguished using T-SNE method.

In [12]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

Problem 2 - Part 2 - Unsupervised Feature Selection

Load & Process Data

In [2]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_unsupervised\\'
X = np.load(path + 'X.npy')
```

In [3]:
```
X_log = np.log2(X+1)

print('Dataset : Gene Expression Matrix')
print('------------------------------------------------------------')
print('(Nb_Cells, Nb_Genes) = ', X.shape, '      (rows x columns)')
print('Min_Value , Max_Value = ', round(np.min(X),0), ',' , round(np.max(X),0))
print('------------------------------------------------------------')
print('Apply log-transform to reduce this extremely stretched range:')
print('X_log <- np.log2(X+1)')
print('Min_Value , Max_Value = ', round(np.min(X_log),0), ',' , round(np.max(X_log),0))
```

```
Dataset : Gene Expression Matrix
------------------------------------------------------------
(Nb_Cells, Nb_Genes) =  (2169, 45768)       (rows x columns)
Min_Value , Max_Value =  0.0 , 341222.0
------------------------------------------------------------
Apply log-transform to reduce this extremely stretched range:
X_log <- np.log2(X+1)
Min_Value , Max_Value =  0.0 , 18.0
```

1. (4 points) Clustering the cells

- At first, let's try and cluster the cells using KMeans.
- The elbow method will help choosing the number of clusters.

If time allows it:

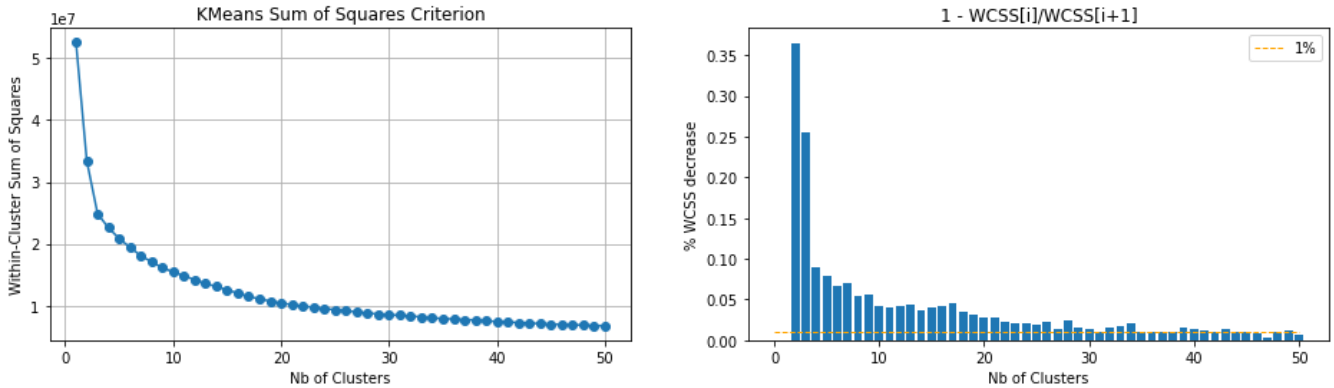
- Silhouette plots will be used to confirm or not the number of clusters
- Other methods will be tried (K-Medoids if outliers happen to distort K-Means results, Gaussian Mixture Model and Hierarchical Clustering) in conjunction with Silhouette plots to choose the best model / number of clusters

KMeans on Top 50 Principal Components

In [4]:
```
pca_log = PCA(random_state = 0)
Z_log = pca_log.fit_transform(X_log)
Z_log_50 = Z_log[:,:50]
```

In [5]:
```
N=50
all_kmeans = [KMeans(n_clusters=i+1,n_init=100, random_state=0) for i in range(N)]
for i in range(N):
    all_kmeans[i].fit(Z_log_50)
```

In [6]:
```
plt.figure(figsize=(16,4))

plt.subplot(1,2,1)
#sklearn: sum of squares = "inertia"
inertias = [all_kmeans[i].inertia_ for i in range(N)]
plt.plot(np.arange(1,N+1),inertias, marker='o')
plt.title("KMeans Sum of Squares Criterion"); plt.xlabel("Nb of Clusters"); plt.ylabel("Within-Clus
ter Sum of Squares")
plt.grid()

plt.subplot(1,2,2)
plt.bar(range(2, N+1), [round((inertias[i]-inertias[i+1])/inertias[i],3) for i in range(N-1)])
plt.hlines(y=0.01, xmin=0, xmax=50, color='orange', linestyles='dashed', linewidth=1, label='1%')
plt.title("1 - WCSS[i]/WCSS[i+1]"); plt.xlabel("Nb of Clusters"); plt.ylabel("% WCSS decrease")
plt.legend()

plt.show()
```



In [7]:
```
print('First Nb of cluster with WCSS decrease of less than 1% (of WCSS)')
print('Nb_Clusters = 30')
print('WCSS_Decrease =', round((inertias[29]-inertias[30])/inertias[29],5))
print('---------------------------')
```

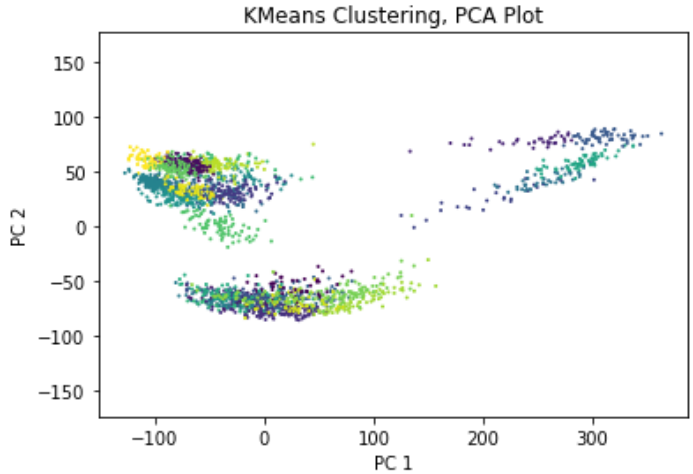
```
First Nb of cluster with WCSS decrease of less than 1% (of WCSS)
Nb_Clusters = 30
WCSS_Decrease = 0.00959
---------------------------
```

- We select N=30 clusters on the basis of this criteria (elbow method, qualitatively guided by the number of clusters used in the evaluation set, namely 35)

Visualization of this clustering choice

In [8]:
```
N=30
kmeans = KMeans(n_clusters=N,n_init=100, random_state = 0)
y = kmeans.fit_predict(Z_log_50)
```

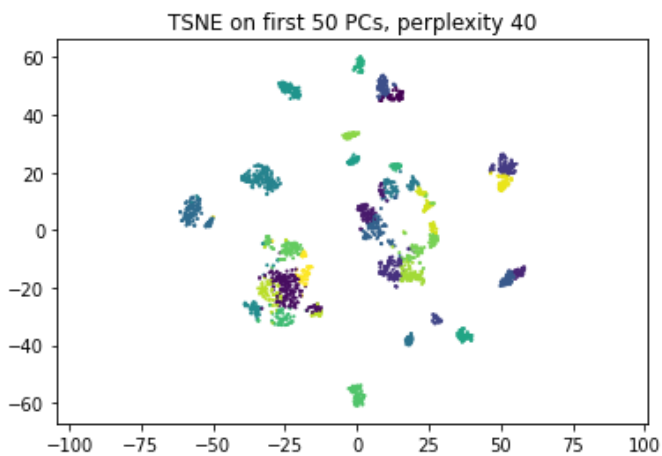
In [9]:
```
plt.scatter(Z_log_50[:,0],Z_log_50[:,1],c=y, s=1)
plt.title("KMeans Clustering, PCA Plot"); plt.xlabel("PC 1"); plt.ylabel("PC 2")
plt.axis("equal")
plt.show()
```



- PCA doesn't represent well this choice for the number of clusters.
- T-SNE, below, is a better option and allows representing these 30 clusters.

In [10]:
```
tsne = TSNE(n_components=2,verbose=1,perplexity=40, random_state=0)
Z_tsne = tsne.fit_transform(Z_log_50)
plt.scatter(Z_tsne[:,0],Z_tsne[:,1], c=y, s=1)
plt.title("TSNE on first 50 PCs, perplexity 40")
plt.axis("equal")
plt.show()
```

```
[t-SNE] Computing 121 nearest neighbors...
[t-SNE] Indexed 2169 samples in 0.037s...
[t-SNE] Computed neighbors for 2169 samples in 0.381s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2169
[t-SNE] Computed conditional probabilities for sample 2000 / 2169
[t-SNE] Computed conditional probabilities for sample 2169 / 2169
[t-SNE] Mean sigma: 20.955963
[t-SNE] KL divergence after 250 iterations with early exaggeration: 55.334656
[t-SNE] KL divergence after 1000 iterations: 0.490110
```



In [84]:
```
import numpy as np
import matplotlib.pyplot as plt
import random
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Reload previous data (should consider pickle...)

In [2]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_unsupervised\\'
X = np.load(path + 'X.npy')
X_log = np.log2(X+1)
```

In [3]:
```
pca_log = PCA(random_state = 0)
Z_log = pca_log.fit_transform(X_log)
Z_log_50 = Z_log[:,:50]
```

In [4]:
```
N=30
kmeans = KMeans(n_clusters=N,n_init=100, random_state = 0)
y = kmeans.fit_predict(Z_log_50)
```

2. (6 points) Logistic regression

Firstly let's split the data into a training and a validation set:

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(X_log, y, test_size=0.2, random_state=0)
```

Then let's perform logistic regression (multi-class = one-vs-rest).

Given that:

- l1 regularization (Lasso) produces sparse models but doesn't handle models with p features >> n observations well;
- l2 regularization (Ridge) can handle this type of datasets but doesn't produce a sparse solution; Elastic net would be the prefered choice of regularization.

But it is computationnaly more expensive, hence we've chosen **l2 regularization**.

In [6]:
```
#logreg = LogisticRegression(C=c, penalty="elasticnet", solver='saga', l1_ratio=0.5, random_state=
0, max_iter=2000, multi_class="ovr")
```

In [9]:
```
test_accuracy={}
print('1/regularization: ', end='')
for c in [0.001, 0.01, 0.1, 1, 10]:
    print(c, end='')
    logreg = LogisticRegression(C=c, penalty="l2", random_state=0, max_iter=2000, multi_class="ovr"
)
    logreg.fit(X_train, y_train)
    test_accuracy[c] = accuracy_score(y_test, logreg.predict(X_test))
    print('(', round(test_accuracy[c],3), ')', end='')
test_accuracy
```

```
1/regularization: 0.001( 0.924 )0.01( 0.924 )0.1( 0.922 )1( 0.924 )10( 0.919 )
```

Out[9]:
```
{0.001: 0.923963133640553,
 0.01: 0.923963133640553,
 0.1: 0.9216589861751152,
 1: 0.923963133640553,
 10: 0.9193548387096774}
```

<u>SOMETHING WRONG... TO REDO...</u>

Regularization results seem wrong. It could be that logistic regression didn't converge.

=> For now we choose C=0.1 and rerun with larger max_iter:

In [10]:
```
logreg = LogisticRegression(C=0.1, penalty="l2", random_state=0, max_iter=20000, multi_class="ovr")
logreg.fit(X_train, y_train)
```

```
0.9216589861751152
```

In [15]:
```
print('Regularization parameter:')
print('C =', 0.1)
print('-----------------------')
print('Validation performance:')
print('Accuracy score:', round(accuracy_score(y_test, logreg.predict(X_test)),5))
```

```
Regularization parameter:
C = 0.1
-----------------------
Validation performance:
Accuracy score: 0.92166
```

<u>3. (9 points) Logistic regression - Top 100 coefficients</u>

In [48]:
```
print('Logistic regression coefficients shape (n_rows <=> nb_of_classes):')
logreg.coef_.shape
```

```
Logistic regression coefficients shape (n_rows <=> nb_of_classes):
```

Out[48]:
```
(30, 45768)
```

Take top 100 coefficients:

In [49]:
```
sum_abs_over_classes = np.sum(np.abs(logreg.coef_), axis=0)
Top100_Genes = sum_abs_over_classes.argsort()[-100:][::-1]
print('Position of top 100 genes:')
Top100_Genes
```

```
Position of top 100 genes:
```

Out[49]:
```
array([36985,  4356, 31421, 41846,  5849,  7473,  5838, 31889, 24590,
       34516, 31662, 36892,  4688, 41265, 42749,  4084,  5551,  5609,
       38779, 34844, 40045, 38729,  4928, 41728, 42363,  5741, 24483,
        6626,  5752,  7897,  8511,  2770, 38286, 38705, 36823, 44094,
        6040, 38332,  4509,  5809, 25442, 40611,  5252, 40043, 24336,
       40160,  4498, 36986, 38467,  8001, 42610,  5523, 45653, 41533,
       37125, 34403, 38605, 41386, 36788, 32424, 41385, 39806,  4832,
       25648, 42218, 24302, 41828, 24592,  3189, 41195,  6951,  7287,
       44937,  4829, 44774, 23404, 38860,  4855,  2560, 43431, 41711,
        8571, 38551,  4331, 37400,  6648, 25448,  4375,  8467, 34900,
        5866,  4354,  3527, 34890,  4441, 32140,  2603, 26116, 38848,
        7643], dtype=int64)
```

In [51]:
```
del X; del X_log; del X_train; del X_test; del y_train; del y_test; del y; del Z_log; del Z_log_50
```

In [80]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_evaluation\\'
X = np.load(path + 'X_train.npy')
X_log = np.log2(X+1)
y_train = np.load(path + 'y_train.npy')

Xt = np.load(path + 'X_test.npy')
Xt_log = np.log2(Xt+1)
y_test = np.load(path + 'y_test.npy')
```

NB: l1 regularization works as n > p for now

In [82]:
```
#Take 100 top genes

#Training
X_train = X_log[:,list(Top100_Genes)]
logreg = LogisticRegressionCV(cv=5,Cs=[0.001,0.01,0.1,1,10],max_iter=5000,penalty="l1",solver="libl
inear",multi_class="ovr", random_state=0)
logreg.fit(X_train,y_train)
print('Training performance:')
print(round(logreg.score(X_train,y_train),5))

#Testing
X_test = Xt_log[:,list(Top100_Genes)]
print('Performance of the trained logistic regression on the evaluation set:')
print(round(logreg.score(X_test,y_test),5))
```

```
Training performance:
0.99721
Performance of the trained logistic regression on the evaluation set:
0.91877
```

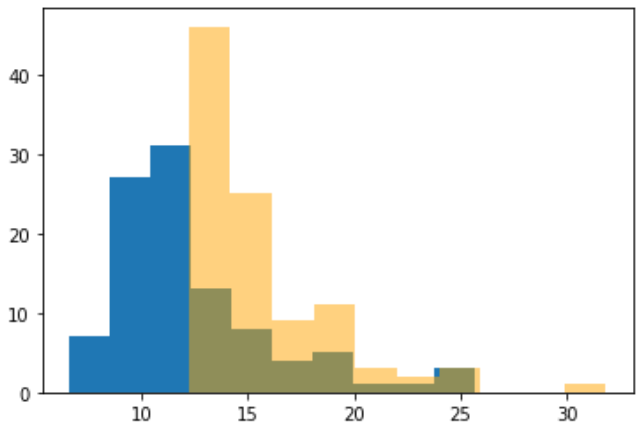
In [133]:
```
#Random selection of 100 genes

list_100_random = random.sample(list(np.arange(X_log.shape[1])), k=100)

#Training
X_train = X_log[:,list_100_random]
logreg_random = LogisticRegressionCV(cv=5,Cs=[0.001,0.01,0.1,1,10],max_iter=5000,penalty="l1",solve
r="liblinear",multi_class="ovr", random_state=0)
logreg_random.fit(X_train,y_train)
print('Training performance:')
print(round(logreg_random.score(X_train,y_train),5))

#Testing
X_test = Xt_log[:,list_100_random]
print('Performance of the trained logistic regression on the evaluation set:')
print(round(logreg_random.score(X_test,y_test),5))
```

```
Training performance:
0.63324
Performance of the trained logistic regression on the evaluation set:
0.47383
```

In [139]:
```
#100 genes with highest variance

Top100_Variance = np.var(X_log, axis=0).argsort()[-100:][::-1]

#Training
X_train = X_log[:,list(Top100_Variance)]
logreg_variance = LogisticRegressionCV(cv=5,Cs=[0.001,0.01,0.1,1,10],max_iter=5000,penalty="l1",sol
ver="liblinear",multi_class="ovr", random_state=0)
logreg_variance.fit(X_train,y_train)
print('Training performance:')
print(round(logreg_variance.score(X_train,y_train),5))

#Testing
X_test = Xt_log[:,list(Top100_Variance)]
print('Performance of the trained logistic regression on the evaluation set:')
print(round(logreg_variance.score(X_test,y_test),5))
```

```
Training performance:
0.99536
Performance of the trained logistic regression on the evaluation set:
0.91516
```

In [157]:
```
plt.hist(np.var(X_log[:,list(Top100_Genes)],axis=0))
plt.hist(np.var(X_log[:,list(Top100_Variance)],axis=0), color='orange', alpha=0.5)
plt.show()
```



=> Our selection based on logistic regression's strongest coefficients includes a subset of the genes with most variance.

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

In [3]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p1\\'
X = np.load(path + 'X.npy')
X_log = np.log2(X+1)
```

Problem 2 - Influence of Hyper-parameters

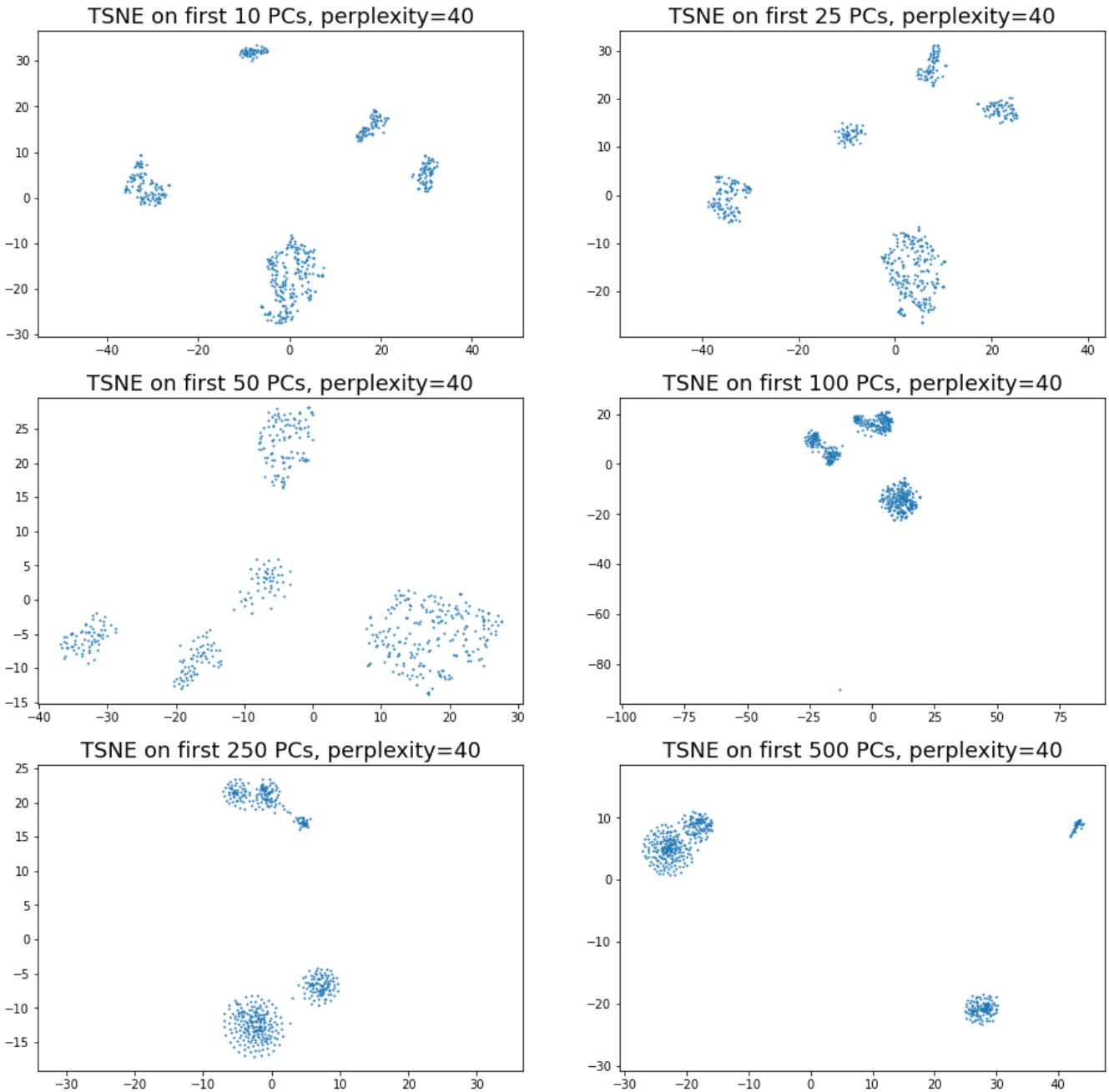
---

1. (3 points) Sensitivity of T-SNE to the number of Principal Components

Let's run T-SNE on an increasing number of PCs.

In [4]:
```
pca_log = PCA()
Z_log = pca_log.fit_transform(X_log)
```

In [6]:
```
plt.figure(figsize=(16,16))

i=0
#for n in [1, 2, 5, 10, 100, 500]:
for n in [10, 25, 50, 100, 250, 500]:
    i+=1
    plt.subplot(3,2,i)
    tsne = TSNE(n_components=2,verbose=0,perplexity=40, random_state=0)
    Z_tsne = tsne.fit_transform(Z_log[:,:n])
    plt.scatter(Z_tsne[:,0],Z_tsne[:,1], s=1)
    plt.title('TSNE on first ' + str(n) + ' PCs, perplexity=40',size=18)
    plt.axis("equal")
plt.show()
```



Increasing the number of principal components:

- brings some clusters together (until the reduction of the number of clusters);
- separates data points within clusters more homogeneously (with high dimension, all points are more likely to be far away from each other).

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
```

2. (13 points) Sensitivity of results to hyper-parameters

Let's analyze the sensitivity of:

A) Visualization of genes subtypes:

- T-SNE perplexity
- T-SNE learning rate

B) Cells clustering:

- Effect of number of PC's chosen

A) Visualization of genes subtypes

Let's revisit the p2_unsupervised dataset and check whether modifying T-SNE perplexity or learning rate changes our conclusions.

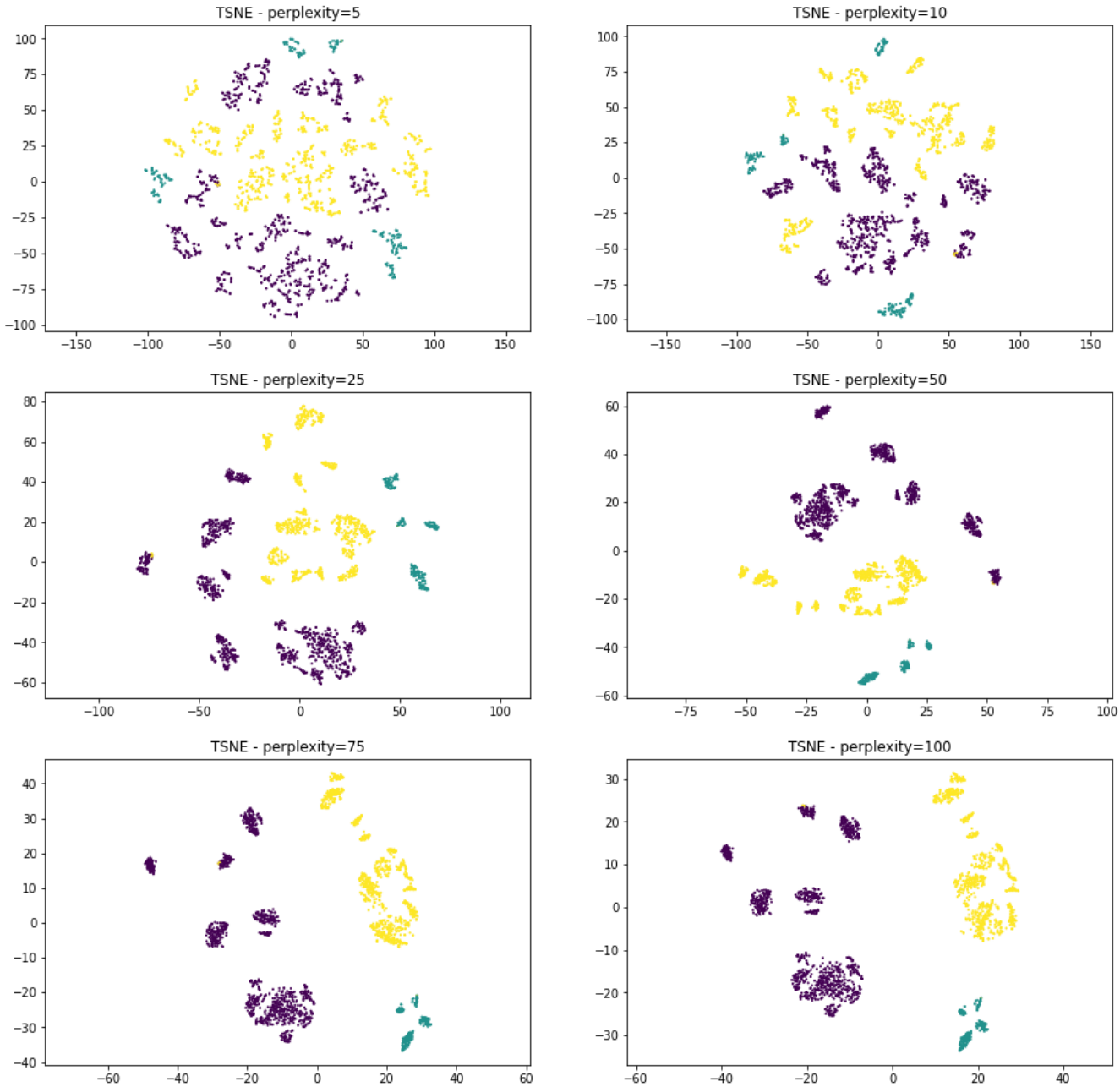
In [2]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_unsupervised\\'
X = np.load(path + 'X.npy')
X_log = np.log2(X+1)
```

In [3]:
```
pca_log = PCA(random_state = 0)
Z_log = pca_log.fit_transform(X_log)
# We keep only 50 PCs
Z_log_50 = Z_log[:,:50]
```

In [4]:
```
# We assign cells to three clusters
kmeans = KMeans(n_clusters=3,n_init=100, random_state = 0)
y = kmeans.fit_predict(Z_log_50)
```

Sensitivity to perplexity

In [12]:
```
plt.figure(figsize=(16,16))
i=0
for p in [5, 10, 25, 50, 75, 100]:
    i+=1
    plt.subplot(3,2,i)
    tsne = TSNE(n_components=2, perplexity=p, learning_rate=200, random_state=0)
    Z_tsne = tsne.fit_transform(Z_log_50)
    plt.scatter(Z_tsne[:,0],Z_tsne[:,1], c=y, s=1)
    plt.title("TSNE - perplexity=" + str(p))
    plt.axis("equal")
plt.show()
```

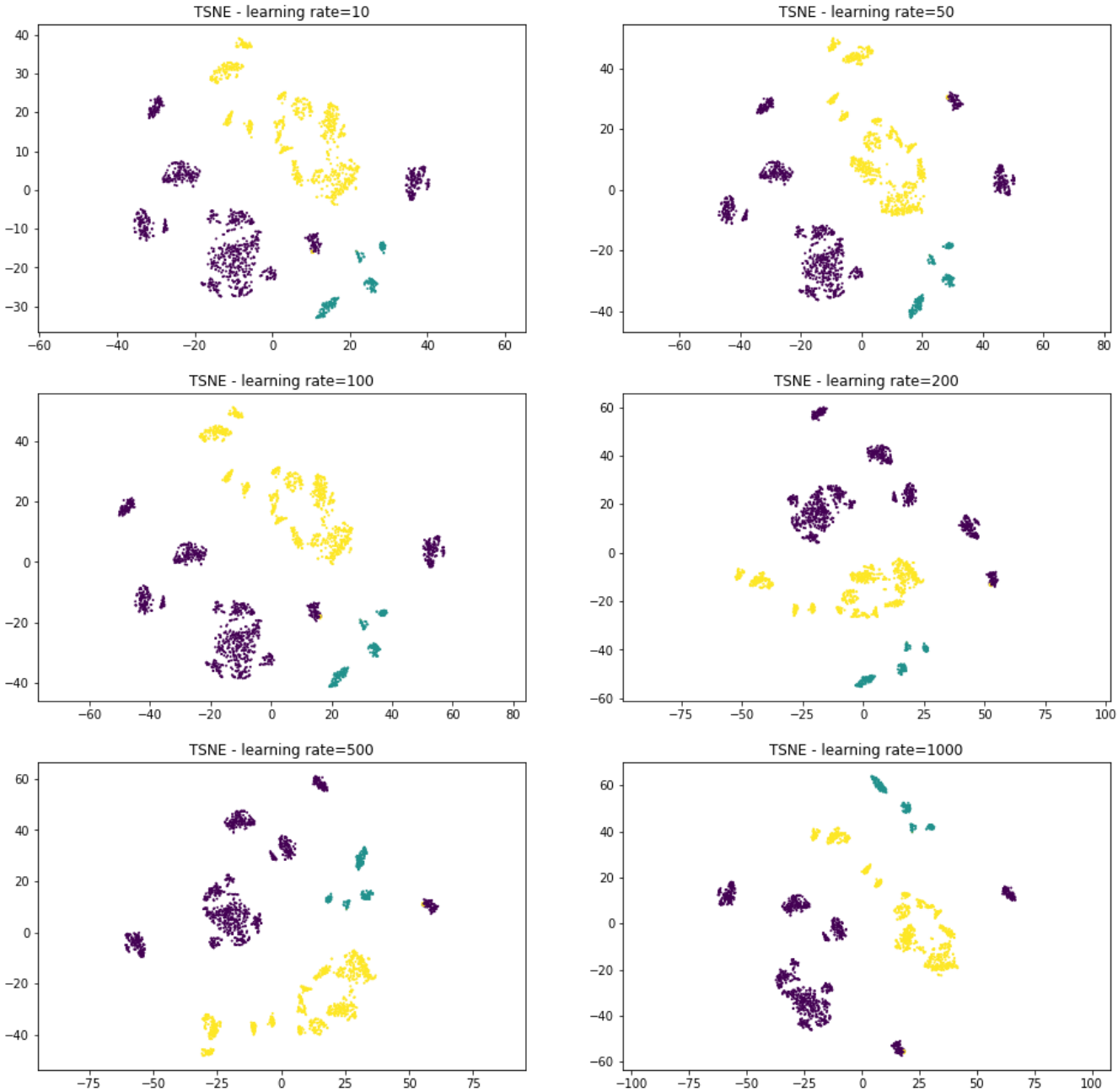


"The perplexity is related to the number of nearest neighbors that is used in other manifold learning algorithms. Larger datasets usually require a larger perplexity." - Scikit-learn documentation

=> Low perplexity values create more smaller clusters of close neighbors. We observe that the number of genes sub-types converges rapidly from perplexity ~25 onwards.

<u>Sensitivity to learning rate</u>

In [13]:
```
# Sensitivity to learning rate

plt.figure(figsize=(16,16))
i=0
for r in [10, 50, 100, 200, 500, 1000]:
    i+=1
    plt.subplot(3,2,i)
    tsne = TSNE(n_components=2, perplexity=50, learning_rate=r, random_state=0)
    Z_tsne = tsne.fit_transform(Z_log_50)
    plt.scatter(Z_tsne[:,0],Z_tsne[:,1], c=y, s=1)
    plt.title("TSNE - learning rate=" + str(r))
    plt.axis("equal")
plt.show()
```

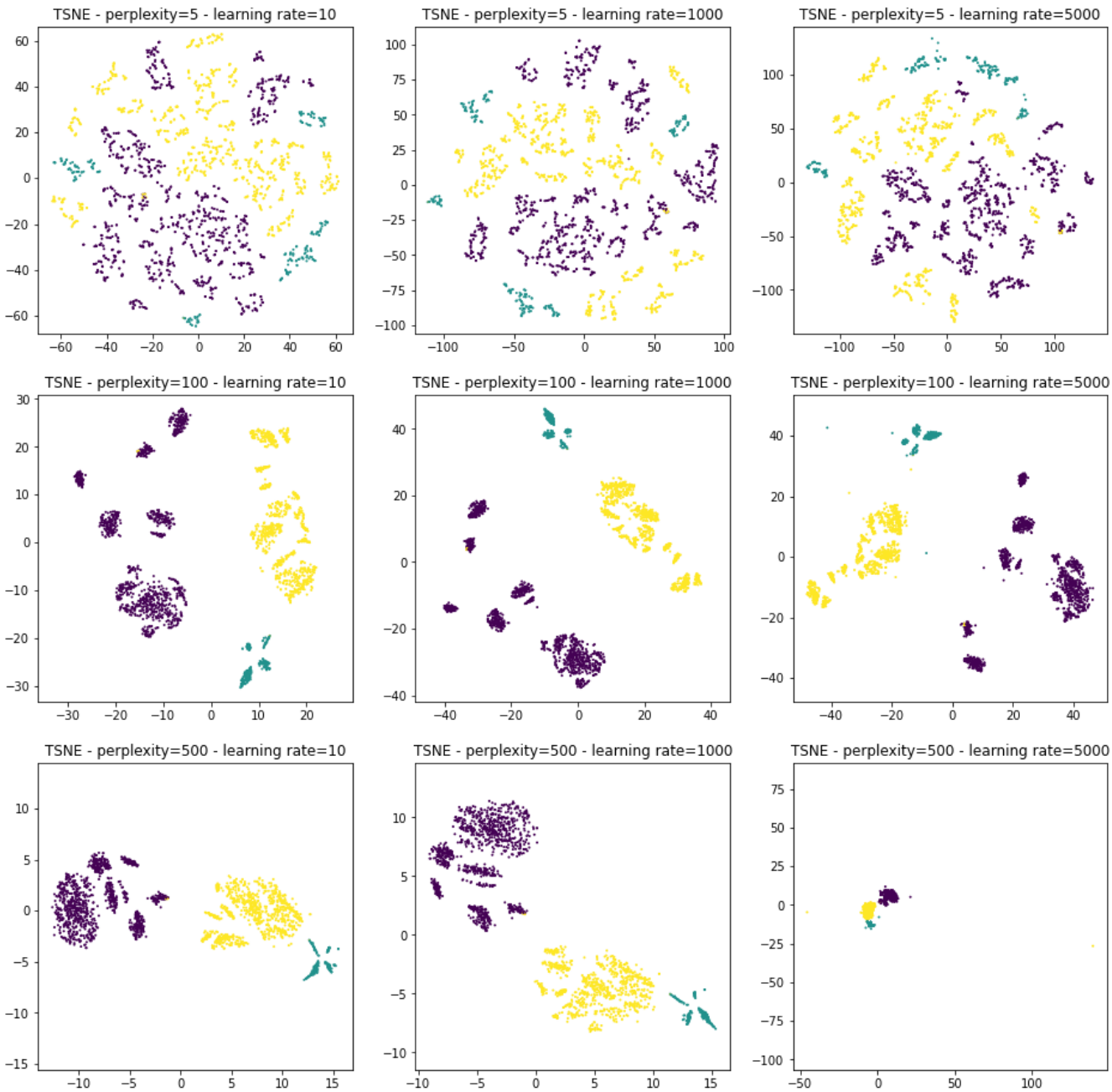


The learning rate controls the step size of the gradient updates. Small learning rate leads to more data being separated, while higher learning rate tends to form more compact clusters.

We actually observe low sensitivity to learning rate on this dataset (for learning rates between 10 and 1000).

Joint sensitivity to perplexity and learning rate

In [18]:
```
plt.figure(figsize=(16,16))
i=0
for p in [5, 100, 500]:
    for r in [10, 1000, 5000]:
        i+=1
        plt.subplot(3,3,i)
        tsne = TSNE(n_components=2, perplexity=p, learning_rate=r, random_state=0)
        Z_tsne = tsne.fit_transform(Z_log_50)
        plt.scatter(Z_tsne[:,0],Z_tsne[:,1], c=y, s=1)
        plt.title("TSNE - perplexity=" + str(p) + " - learning rate=" + str(r))
        plt.axis("equal")
plt.show()
```



Interestingly, joint effects of perplexity and learning rate are only observable for very large joint values of these hyperparameters.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

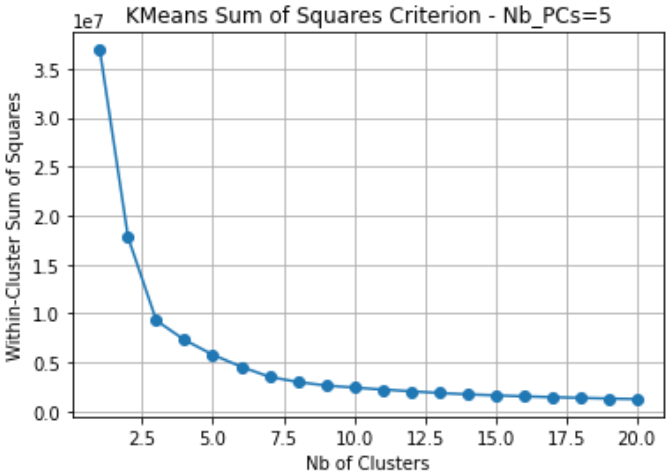
B) Cells clustering

Let's revisit the p2_unsupervised dataset and look at the effect of the number of PCs chosen on clustering.

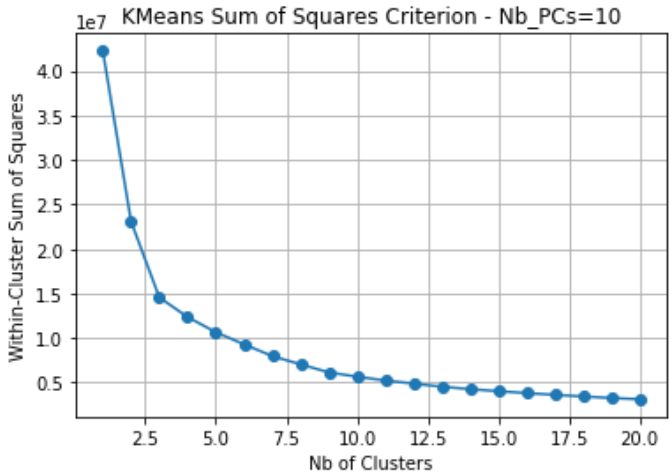
In [2]:
```
path = r'C:\Users\phili\Desktop\Data Analysis\HW2\data\p2_unsupervised\\'
X = np.load(path + 'X.npy')
X_log = np.log2(X+1)
```

In [3]:
```
pca_log = PCA(random_state = 0)
Z_log = pca_log.fit_transform(X_log)
```

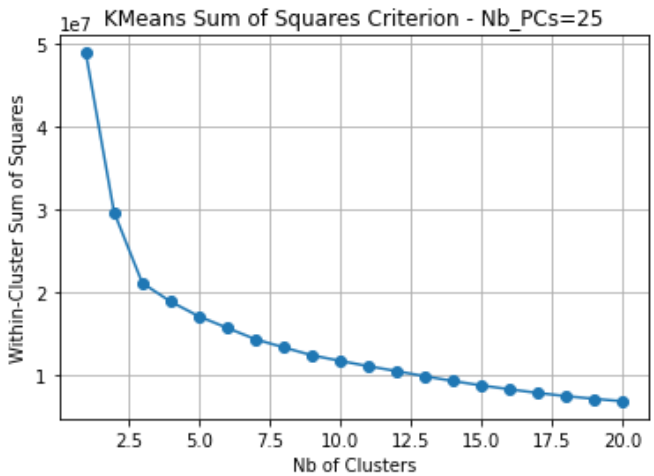
In [9]:
```
#Nb_PCs
nb_pc = 5

Z_log_nb_pc = Z_log[:,:nb_pc]
N=20
all_kmeans = [KMeans(n_clusters=i+1,n_init=100, random_state=0) for i in range(N)]
for i in range(N):
    all_kmeans[i].fit(Z_log_nb_pc)

inertias = [all_kmeans[i].inertia_ for i in range(N)]
plt.plot(np.arange(1,N+1),inertias, marker='o')
plt.title("KMeans Sum of Squares Criterion - Nb_PCs=" + str(nb_pc)); plt.xlabel("Nb of Clusters");
plt.ylabel("Within-Cluster Sum of Squares")
plt.grid()
```



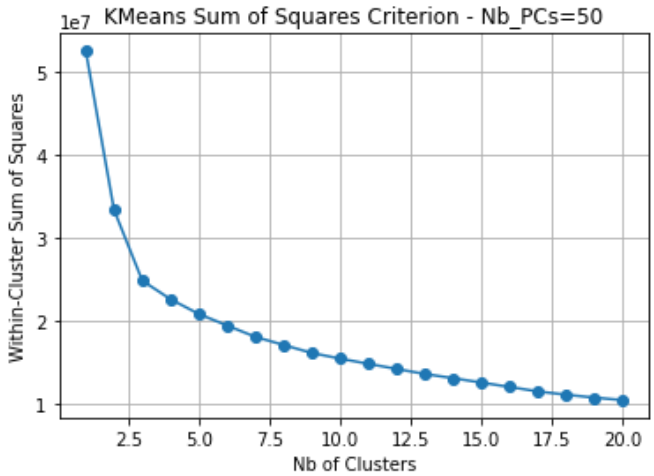
In [10]:
```
nb_pc = 10

Z_log_nb_pc = Z_log[:,:nb_pc]
N=20
all_kmeans = [KMeans(n_clusters=i+1,n_init=100, random_state=0) for i in range(N)]
for i in range(N):
    all_kmeans[i].fit(Z_log_nb_pc)

inertias = [all_kmeans[i].inertia_ for i in range(N)]
plt.plot(np.arange(1,N+1),inertias, marker='o')
plt.title("KMeans Sum of Squares Criterion - Nb_PCs=" + str(nb_pc)); plt.xlabel("Nb of Clusters");
plt.ylabel("Within-Cluster Sum of Squares")
plt.grid()
```



In [11]:
```
nb_pc = 25

Z_log_nb_pc = Z_log[:,:nb_pc]
N=20
all_kmeans = [KMeans(n_clusters=i+1,n_init=100, random_state=0) for i in range(N)]
for i in range(N):
    all_kmeans[i].fit(Z_log_nb_pc)

inertias = [all_kmeans[i].inertia_ for i in range(N)]
plt.plot(np.arange(1,N+1),inertias, marker='o')
plt.title("KMeans Sum of Squares Criterion - Nb_PCs=" + str(nb_pc)); plt.xlabel("Nb of Clusters");
plt.ylabel("Within-Cluster Sum of Squares")
plt.grid()
```



In [12]:
```
nb_pc = 50

Z_log_nb_pc = Z_log[:,:nb_pc]
N=20
all_kmeans = [KMeans(n_clusters=i+1,n_init=100, random_state=0) for i in range(N)]
for i in range(N):
    all_kmeans[i].fit(Z_log_nb_pc)

inertias = [all_kmeans[i].inertia_ for i in range(N)]
plt.plot(np.arange(1,N+1),inertias, marker='o')
plt.title("KMeans Sum of Squares Criterion - Nb_PCs=" + str(nb_pc)); plt.xlabel("Nb of Clusters");
plt.ylabel("Within-Cluster Sum of Squares")
plt.grid()
```



The more principal components we select, the higher the number of clusters.

For instance, a WCSS of say 0.5e7 is reached:

- For 5 PCs: ~5 clusters
- For 10 PCs: ~11 clusters
- For 20 PCs: ~20 clusters

=> Domain knowledge can be of help in defining the number of PCs, hence clusters